



igor_suhorukov
19 мая в 06:11

Некорпоративный Хабр: семантический поиск и фильтрация по структурированным полям в браузере (llama.cpp в WebAssembly)

🔗 Средний ⌚ 16 мин 👁 11K

Искусственный интеллект, Поисковые технологии*, Базы данных*

Классический RAG индексирует исходный текст документа, предварительно разбивая на фрагменты. Потом рассчитывает векторное представление фрагментов и сохраняет их векторные представления в базу данных для поиска. Это дает возможность искать по сходству фрагментов текста и поискового запроса пользователя, но не дает возможность искать по более высокоуровневому резюме и смыслу, темам поднятым в тексте и прочему. Также не помогает с аналитикой по содержанию.

Бесплатный проект text-metadata-generator позволяет выполнять запросы к LLM по каждому документу из коллекции документов, результаты вывода LLM проверяются по JSON схеме.

Зачем может пригодиться эта программа и подход со структурированием текстовой информации

- своя библиотека с каталогом - поиск по локальным документам с использованием комбинации SQL предикатов и семантического поиска
- аналитика по документам, возможность находить новое в текстах: комбинируя структурированные поля созданные LLM из исходного текста, и находя закономерности с уже существующими в документе метаданными. Например, связывая с рейтингом признак NSFW, тон повествования, полноту содержания итп.
- разгрести "авгиевы конюшни" личных заметок в Obsidian или git репозитории с Markdown файлами

Поиск по информации статей реализован в виде веб-приложения на llama.cpp в WebAssembly (wllama) для расчета эмбедингов и Wasm сборке колоночной базы данных DuckDB. Получился статический веб сайт с JavaScript которому для работы не нужен бэкэнд для поиска данных и запуска embeddinggemma, полный serverless!

Рассмотрим как работает данный подход на 13275 статьях с Хабра, а также текстах трех песен...

Примеры обработки данных

Технические статьи это главный источник для которого разрабатывался данный подход, но можно запускать анализ данных на текстах популярных песен. Если для одного и того же текста использовать разные LLM модели, то можно получить синтетический плюрализм "мнений" от LLM

▸ [Nautilus Pompilius: Воздух](#)

▸ [Jonathon Coulton: Still Alive](#)

РЕКЛАМА



АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, рейв от ЛАУД



► The Beatles: Yesterday

4 rows (filters: is NSFW-No)

Filters

Record limit: 5 15 30 50 100 99999 Reset all Expand

Enter a query

Describe what you're looking for... Search Clear

Results (4)

ID	Original Text	Similarity score	URL
1	Текст описывает сюрреалистическую сцену противостояния одного человека и вооруженного отряда. Главный герой совершает не...	—	View
2	Лирический текст о глубоком сожалении, утрате любви и тоске по прошлому, когда жизнь казалась проще и счастливее.	—	View
3	Текст представляет собой ироничную лекцию от лица искусственного интеллекта (GlaDOS) из игры Portal. Персонаж поздравляет...	—	View
4	Автор описывает процесс создания системы индексации текстов, которая дополняет классический RAG структурированными метад...	—	View

Log

Clear log

```
[6:06:01 AM] Initializing DuckDB-WASM.  
[6:06:01 AM] DuckDB-WASM ready  
[6:06:02 AM] Loading indexed text table  
[6:06:04 AM] Registered indexed text table  
[6:06:04 AM] Fetching text_embeddings partition files...  
[6:06:05 AM] Registered 43 partition files  
[6:06:05 AM] Loading text_embeddings table  
[6:06:06 AM] Registered text_embeddings table (flat partitions)  
[6:06:06 AM] Loading filter values...  
[6:06:06 AM] Loading model: embeddingggemma-300m-Q4_0_gguf (Q4_0) (via wllama)...  
[6:06:06 AM] Filter values loaded  
[6:06:09 AM] Model loaded  
[6:06:39 AM] Browsing with filters (no embedding)  
[6:06:39 AM] Found 4 rows
```

Project description

Как получал данные для индексации

Была у меня коллекция статей в формате JSON за 2023 год больше 13тыс шт. (без постов из корпоративных блогов), в атрибуте “textHtml” которых находятся HTML-код статьи. Имя каждого файла - идентификатор публикации плюс “.json”. Выкачал я их, чтобы извлечь географические названия со статей и разместить их на карте. Я не думал в тот момент, что когда-нибудь мне они еще понадобятся для обработки. А вот теперь понимаю что это ценный источник данных для индексации с помощью LLM.

Перед обработкой LLM - полезно использовать `randsos` для конвертации HTML в Markdown. Одновременно обогатил каждый .md файл заголовком с метаданными (названием, датой публикации, автором, рейтингом итп)

► bash для превращение json публикации в markdown

Использовал свою freeware утилиту [text-metadata-generator](#) для обработки текстов с помощью LLM и сохранения структурированных данных и рассчитанных эмбеддингов в DuckDB.



АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, ревью от ЛАУД

Content description and metadata generator

DashboardTask BatchesSettings

Create BatchPause ExecutionRefreshMetadata Viewer

ID	Name	Description	Batch Status	Priority	Batch URL	Created	Updated
1	Index directory /home/iam/D...			0	File/home/iam/Documents/articl...	2026-05-02 18:08	2026-05-02 18:09

Batch (ID: 1): Index directory /home/iam/D...

285 hours remainWaiting For Processing: 13125 | Processing: 61 | Completed: 89

Filter by Status: All Statuses

Clear FilterRefresh

ID	Status	Priority	URL	Processing Time	Created	Updated	Details
1		0	File/home/iam/Documents/articles/...	148786	2026-05-02 18:08	2026-05-02 18:11	
2		0	File/home/iam/Documents/articles/...	97353	2026-05-02 18:08	2026-05-02 18:11	
3		0	File/home/iam/Documents/articles/...	75355	2026-05-02 18:08	2026-05-02 18:10	
4		0	File/home/iam/Documents/articles/...	37503	2026-05-02 18:08	2026-05-02 18:10	
5		0	File/home/iam/Documents/articles/...	123489	2026-05-02 18:08	2026-05-02 18:11	
6		0	File/home/iam/Documents/articles/...	67380	2026-05-02 18:08	2026-05-02 18:10	
7		0	File/home/iam/Documents/articles/...	0	2026-05-02 18:08	2026-05-02 18:09	
8		0	File/home/iam/Documents/articles/...	70710	2026-05-02 18:08	2026-05-02 18:10	
9		0	File/home/iam/Documents/articles/...	78474	2026-05-02 18:08	2026-05-02 18:11	
10		0	File/home/iam/Documents/articles/...	27444	2026-05-02 18:08	2026-05-02 18:10	
11		0	File/home/iam/Documents/articles/...	53277	2026-05-02 18:08	2026-05-02 18:09	
12		0	File/home/iam/Documents/articles/...	95788	2026-05-02 18:08	2026-05-02 18:11	
13		0	File/home/iam/Documents/articles/...	80167	2026-05-02 18:08	2026-05-02 18:10	
14		0	File/home/iam/Documents/articles/...	68703	2026-05-02 18:08	2026-05-02 18:10	
15		0	File/home/iam/Documents/articles/...	59021	2026-05-02 18:08	2026-05-02 18:10	
16		0	File/home/iam/Documents/articles/...	97602	2026-05-02 18:08	2026-05-02 18:11	
17		0	File/home/iam/Documents/articles/...	82867	2026-05-02 18:08	2026-05-02 18:11	
18		0	File/home/iam/Documents/articles/...	129143	2026-05-02 18:08	2026-05-02 18:11	
19		0	File/home/iam/Documents/articles/...	52409	2026-05-02 18:08	2026-05-02 18:10	
20		0	File/home/iam/Documents/articles/...	59021	2026-05-02 18:08	2026-05-02 18:10	
21		0	File/home/iam/Documents/articles/...	75164	2026-05-02 18:08	2026-05-02 18:10	

Приведу здесь для примера сокращенную JSON схему, использованную для структурирования статей:

JSON schema

Content description and metadata generator

DashboardTask BatchesSettings

UI Language: EnglishRefresh

Application Settings

Thread Pool Size: 30Tasks To Fetch: 30

Use Slow Validation Checks:

Component Configuration Settings

Data Processors

DataProcessor (com.github.isuhorukov:text:1.0)

Process text analysis with Ollama/OpenAI LLM to generate structured metadata.

You need Ollama installed locally: https://ollama.com/download after install search LLM model in https://ollama.com/library and download models before start using of this application with command, for example: ollama pull qwen3-vl-8b-instruct-q4_K_M

Another option is to use OpenAI compatible cloud OpenRouter with secret key https://openrouter.ai/workspaces/default/keys and positive account balance https://openrouter.ai/settings/credits

provider: Ollama

Ollama think: false

Ollama numCtx: 40000

url: http://localhost:11434

model: qwen3.5:9b

api-key:

timeout: 360

max-tokens: 8000

Embedding Processors

EmbeddingProcessor (com.github.isuhorukov:text:1.0)

Extract embeddings from structured data.

embedding provider: Ollama

url: http://localhost:11434

model: hf.co/unsloth/embeddinggemma-300m-GGUF-Q4_0

api-key:

timeout: 360

System Information

GPU 0: (NVIDIA GeForce RTX 5090)

Thermal Settings

Application Profiles

nvidia-settings Configuration

PRIME Profiles

Resources

Processes

File Systems

CPU

CPU1 47.5% CPU2 49.0% CPU3 54.5% CPU4 51.5% CPU5 54.0% CPU6 63.3% CPU7 61.6%

CPU8 56.6% CPU9 49.0% CPU10 56.4% CPU11 51.0% CPU12 60.6% CPU13 61.4% CPU14 72.4%

CPU15 71.3% CPU16 61.0% CPU17 80.8% CPU18 78.0% CPU19 74.0% CPU20 82.0% CPU21 78.8%

CPU22 74.5% CPU23 70.7% CPU24 68.7% CPU25 82.0% CPU26 79.2% CPU27 86.9% CPU28 81.8%

CPU29 78.0% CPU30 68.7% CPU31 75.2% CPU32 79.0%

Memory and Swap

Memory 16.4 GB (25.7%) of 63.8 GB Cache 39.3 GB

Swap 0 bytes (0.0%) of 2.1 GB

Network

Receiving 1.3 MiB/s Total Received 2.7 GiB

Sending 116.4 KiB/s Total Sent 527.0 MiB

Disk

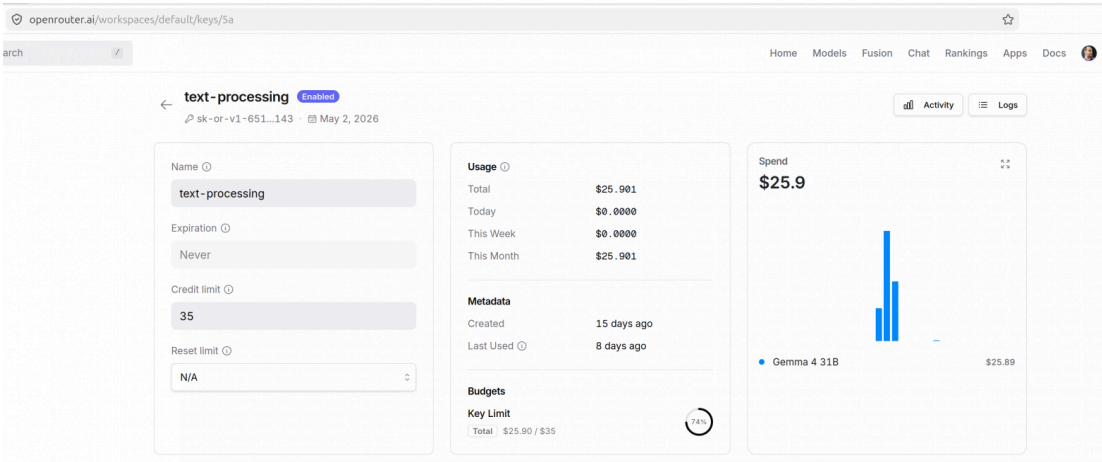
Reading 0 bytes/s Total Read 31.4 GiB

Writing 14.4 MiB/s Total Written 30.0 GiB

Потратил на обработку в 30 параллельных потоков в сумме 26\$ с личного счета в openrouter на статьи.



АЛЬФА ВАЙБ МИТАП
Техно-шоу про ИИ, рейв от ЛАУД



Но эмбединги потом пересчитывал в embeddinggemma локальной ollama. Для расчетов эмбедингов использовал "hf.co/unsloth/embeddinggemma-300m-GGUF:Q4_0". Выбрал я ее для того чтобы расчеты были одинаковыми и в ollama и в wllama.

▸ [CREATE TABLE indexed_text](#)

После завершения обработки выгрузил данные командой

```
duckdb ./data/indexer_text.db
```

Объяснить с

в indexed_text.parquet файл:

```
copy (select task_id,url,title,summary,meaning,target_audience,genre,topics,themes,key_insights
```

Объяснить с

▸ [и эмбединги в 43 файла data \\${i}.parquet](#)

Посчитал токены на входе и выходе нейронок на основе метаданных получилось 111млн входных токенов и 14млн выходных для 13275 статей.

```
select count(*) count, sum(metadata_int['llm.main.inputTokenCount']) input_token, sum(metadata_
```

Объяснить с

count	input_token	output_token
13275	111304592	14743366

Объяснить с

Проблемы при реализации



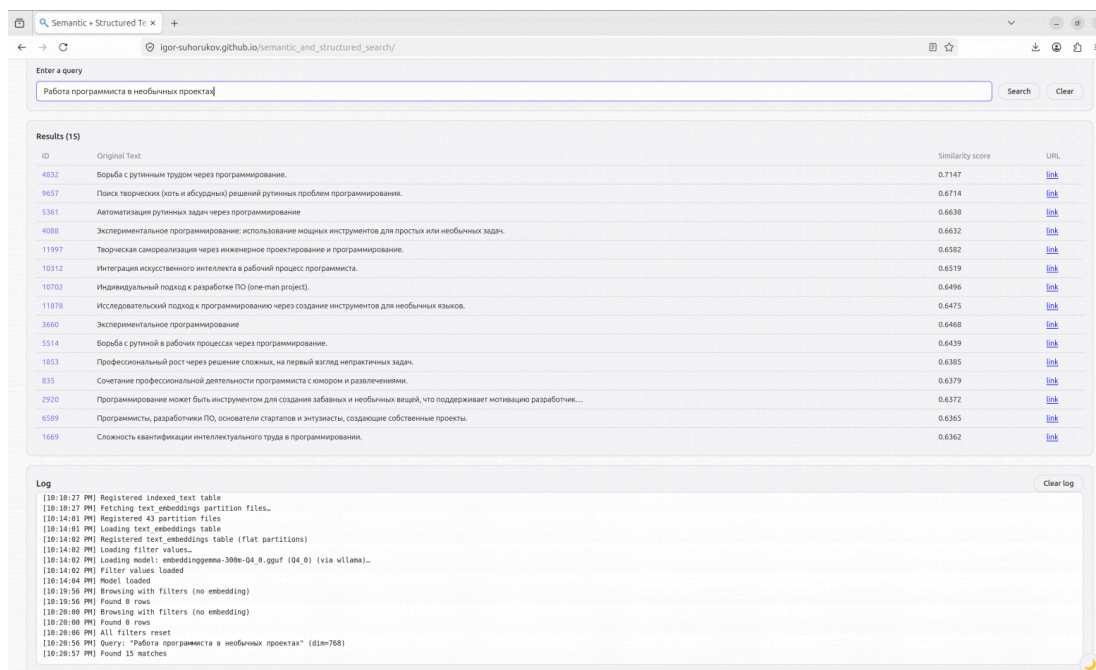
АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, рейв от ЛАУД

Transformers.js + onnx-community/embeddinggemma-300m-ONNX показало что эти эмбединги имеют большую дистанцию и семантический поиск не работает. Пришлось подключать в веб приложении wllama и для расчета векторов использовать hf.co/unsloth/embeddinggemma-300m-GGUF:Q4_0 и в ollama и в wllama. Только после этого семантический поиск начал работать как ожидалось. Также хотелось вначале использовать модель для эмбедингов qwen3-embedding-4b/8b, но отказался по причине потенциальной проблемы производительности модели в wllama и огромного объема эмбедингов после индексации, даже после компрессии ZSTD в parquet.

Особенности реализации веб интерфейса:

Работа semantic_and_structured_search приложения в браузере с WASM duckdb и wllama.

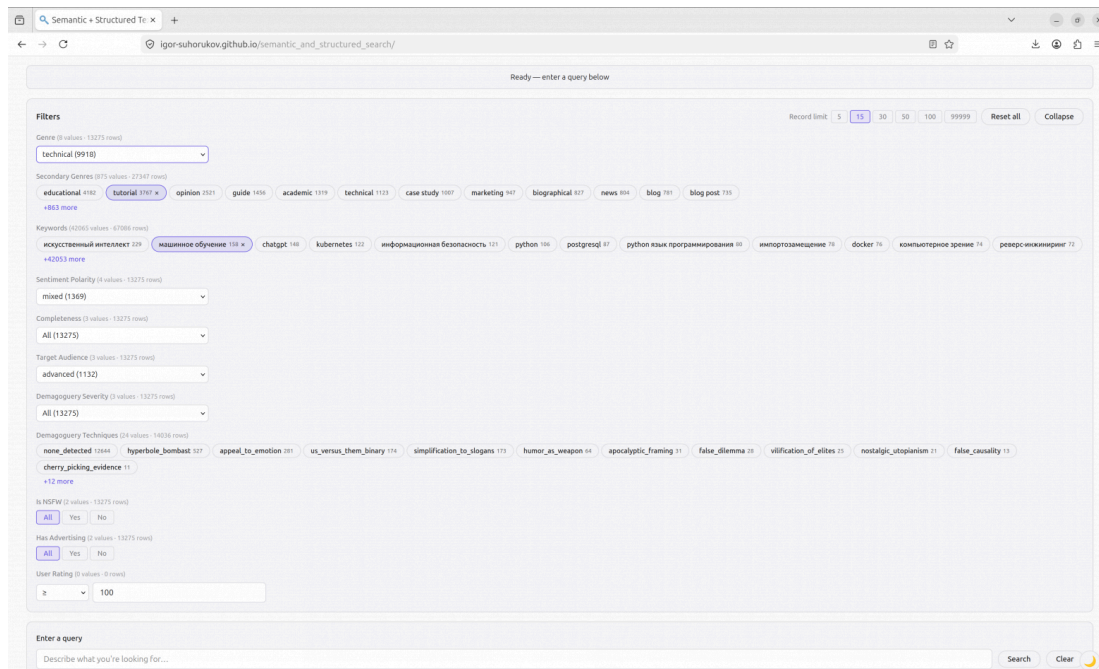


- контроль своих данных: поиск работает в браузере, без отправки на сервер/в облачный API. wllama с EmbeddingGemma 300M и DuckDB-WASM выполняются локально
- векторный поиск по нескольким полям документа - эмбединги вычисляются по summary, meaning, темам, ключевым инсайтам, противоречиям из текста, описанию целевой аудитории
- возможность фильтрации без семантического соответствия - просматривая выборки по жанрам, темам, рейтингу, ключевым словам
- многоуровневая фильтрация: выпадающие списки, множественный выбор, переключатели с тремя состояниями и числовой фильтр, объединяемые как предикаты в запросе через AND и настраиваемый лимит результатов (5/15/50/100 шт.)
- тёмная и светлая темы с сохранением предпочтения в localStorage
- логирование ключевых операций веб приложения - встроенная консоль с логами и цветовой индикацией ошибок
- поддержка структурированных DuckDB-типов: работа с вложенными данными в STRUCT, данными в MAP и массивами
- привычное масштабируемое хранилище - данные хранятся в Parquet-файлах
- в будущем можно добавить новые структурированные колонки в таблицу и новые поля для векторного поиска, не переписывая с нуля приложение



АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, ревью от ЛАУД



Большая часть этого интерфейса сгенерирована моделью qwen3.6 с ручными правками после. На что потратил около 12\$ на токены и пару дней своего времени. Ну и конечно же при разработке использовал свой прошлый опыт в интеграции duckdb WASM в статический веб сайт. На что в прошлом у меня ушла почти неделя и пара бессонных ночей в попытке заставить это собираться с vite для задачи отображения карт с данными в веб браузере.

Можно запустить приложение и с Github Pages, но я не рекомендую этот вариант потому что скачаете почти гигабайт интернет трафика и все равно будет медленно работать. При старте приложению нужно подключение к интернет, чтобы скачать duckdb wasm и llama с CDN и модель эмбедингов с huggingface. Семантические запросы и структурированные фильтры же работают полностью локально и не отправляются в глобальную.

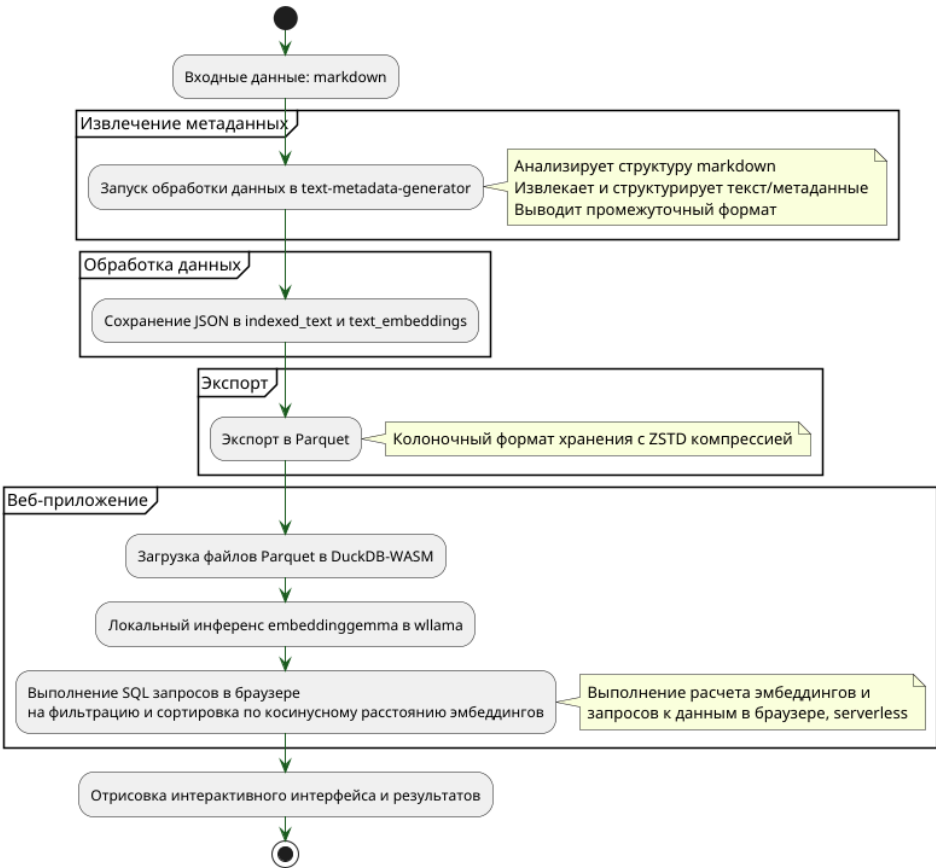
Можете повторить все то же на любых данных

1. Подготовить директорию с вашими файлами в формате plain text, Markdown или HTML.
2. Установить [text-metadata-generator](#)
3. Запустить обработку для директории из пункта 1
4. Остановить приложение. Подключиться к duckdb базе данных в консоли и экспортировать таблицу indexed_text в Parquet
5. Склонировать git репозиторий semantic_and_structured_search и положить в /assets свой indexed_text.parquet
6. Положить в /text_embeddings_partitions веб-приложения файлы с эмбедингами data_0.parquet ... data_42.parquet
7. Запустить локальный веб сервер (например с помощью команды `python3 -m http.server 5000`) и открыть веб приложение в браузере

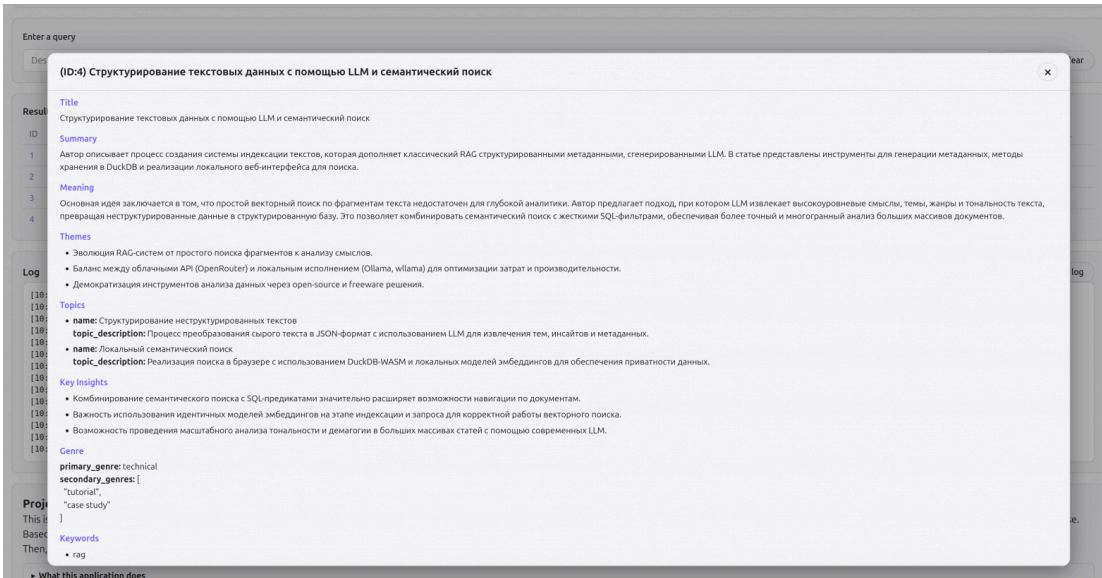


АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, рейв от ЛАУД

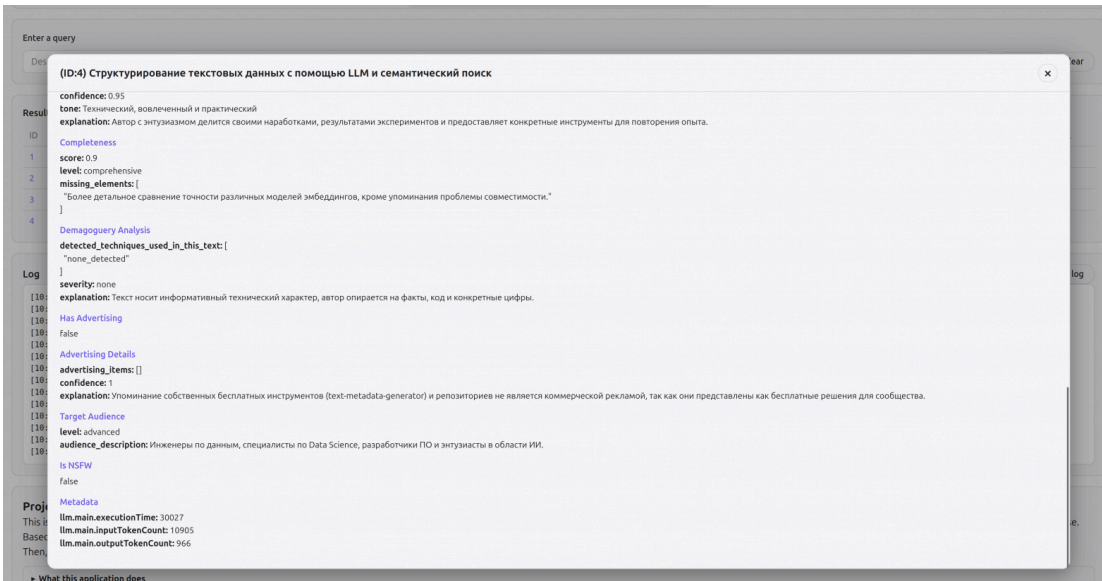
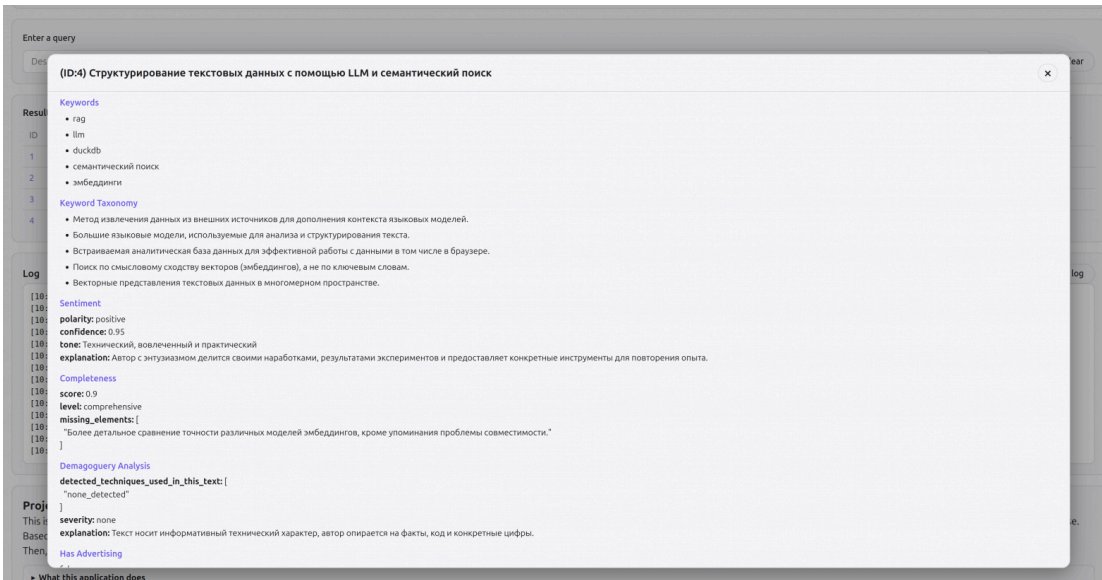


А я проверю работу приложения на исходном тексте этой статьи:



АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, реив от ЛАУД



Интересные находки из проиндексированных статей Хабра

Такой разбор материала позволяет взглянуть по новому на материалы авторов и предпочтения читателей. Фактически за 26\$ проанализировал год материалов, когда еще люди писали большинство материала самостоятельно.

Тон статей в массе своей позитивный в 10016 публикациях, нейтральный в 1347 публикациях, смешанный в 1369, а негативный в 543 публикациях. Небезопасными для работы (NSFW) являются по мнению Gemma4-31b - 88 статей, с аргументацией в чем же проблема. Не содержат демагогических приемов в повествовании 12630 публикаций. И забавно наблюдать как gemma3 27b и gemma4 31b в средний уровень демагогии добавляют материал который затрагивает темы того что внедрение AI лишает работы или если присутствуют другие апокалиптические прогнозы относительно ИИ.

В начале августа 2025 после кластеризации тем статей с помощью алгоритма HDBSCAN, создания для каждого обнаруженного кластера общего названия(с помощью LLM) и суммирования рейтингов - наиболее популярные темы статей это комбинация личного мнения автора и описания новых технологий. Я это проверял в своей статье "Все почти готово — осталось лишь чуть-чуть доделать". Похоже что эти темы действительно откликаются у нас всех.



АЛЬФА ВАЙБ МИТАП
Техно-шоу про ИИ, рейв от ЛАУД

Вы можете попробовать это веб приложение локально или на Github Pages и поиграться фильтрами и семантическим поиском. У меня даже появилось несколько анекдотов на основе этих данных, но по соображениям приличия, предлагаю вам найти их самостоятельно.

Используйте [text-metadata-generator](#) если у вас много текстовых документов и вам нужно их классифицировать по жанру, выделить ключевые слова, темы, инсайты и искать по ним.

Теги: семантический поиск, структурирование информации, локальный поиск по документам, эмбединги, duckdb wasm, wllama, llm-приложения, sql, rag, llm

Хабы: Искусственный интеллект, Поисковые технологии, Базы данных

+7 16 2

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



8K+

176

Охват за 30 дней Карма

Игорь @igor_suhorukov

AI developer / Java / PostgreSQL / Open Source

Подписаться



Комментарии 2



empenoso

19 мая в 10:30

Классификация именно текстовых документов?

Формат имеет значение?



Ответить



igor_suhorukov

19 мая в 12:11

Классификация именно текстовых документов?

В этой статье текстовых. Есть shareware проект для изображений, но это немного другая тема!

Формат имеет значение?

Сейчас поддерживается plain text, Markdown, HTML



Ответить



АЛЬФА ВАЙБ МИТАП

Техно-шоу про ИИ, рейв от ЛАУД